Universidade Federal do Estado do Rio de Janeiro
Centro de Ciências Exatas e Tecnológicas
Escola de Informática Aplicada

Análise Experimental da Privacidade Diferencial no Treinamento de Redes Neurais Profundas

Breno Figueiredo Borges

Rio de Janeiro, RJ – Brasil
Agosto, 2025

Borges, Breno Figueiredo

B732a Análise Experimental da Privacidade Diferencial no Treinamento de Redes Neurais Profundas – Breno Figueiredo Borges – Rio de Janeiro, 2025.

53

Orientador: Pedro Nuno de Souza Moura.

Trabalho de Conclusão de Curso (Graduação) - Universidade Federal do Estado do Rio de Janeiro, Graduação em Sistemas de Informação, 2025.

1. Privacidade Diferencial. 2. Aprendizagem Profunda. I. de Souza Moura, Pedro Nuno, orient. II. Análise Experimental da Privacidade Diferencial no Treinamento de Redes Neurais Profundas

Análise Experimental da Privacidade Diferencial no Treinamento de Redes Neurais Profundas

(Breno Figueiredo Borges)

Projeto de Graduação apresentado à Escola de Informática Aplicada da Universidade Federal do Estado do Rio de Janeiro (UNIRIO) para obtenção do título de Bacharel em Sistemas de Informação.

Aprovada por:

Pedro Nuno de Souza Moura - UNIRIO

Jefferson Elbert Simões - UNIRIO

Geiza Maria Hamazaki da Silva - UNIRIO

Rio de Janeiro, RJ – Brasil

Agosto de 2025

Agradecimentos

Agradeço, em primeiro lugar, à minha mãe, Liliane Figueiredo da Silva, por ser um exemplo de força, dedicação e inspiração em minha vida.

À minha família, pelo apoio incondicional e pelo incentivo constante ao longo de toda a minha trajetória acadêmica.

Ao meu orientador, Pedro Nuno de Souza Moura, pela orientação e contribuições fundamentais para a realização deste trabalho.

Aos meus amigos, que estiveram ao meu lado nos momentos de dificuldade, ouvindo meus desabafos e me oferecendo apoio e conforto durante a jornada da elaboração deste TCC.

Um agradecimento especial ao meu amigo Igor Xavier, pela ajuda na preparação dos slides para a defesa deste TCC.

A todos que, de alguma forma, contribuíram para a realização deste trabalho, deixo aqui minha gratidão.

Resumo

A sociedade contemporânea é marcada pela intensa valorização e exploração de dados, impulsionada pelo avanço das tecnologias de inteligência artificial e, especialmente, pela aprendizagem profunda. Modelos baseados em redes neurais são capazes de extrair padrões complexos e promover inovações significativas em diversas áreas da sociedade. No entanto, esse cenário também evidencia desafios éticos e legais relacionados à privacidade dos indivíduos, uma vez que muitos desses modelos são treinados utilizando grandes volumes de dados sensíveis.

A Privacidade Diferencial surge como um paradigma teórico e prático fundamental para mitigar os riscos associados ao vazamento de informações pessoais. Aplicada ao contexto da aprendizagem profunda, ela visa garantir que a inclusão ou remoção de um único dado não altere significativamente o comportamento do modelo, protegendo assim a privacidade dos indivíduos mesmo em cenários de adversários com grande capacidade de inferência.

Este trabalho se propõe a investigar a aplicação da Privacidade Diferencial no treinamento de modelos de redes neurais profundas, utilizando o algoritmo DP-SGD (Descida de Gradiente Estocástico com Privacidade Diferencial). Para tanto, foram conduzidos experimentos com duas arquiteturas de diferentes complexidades sobre o conjunto de dados MNIST, comparando o desempenho com e sem a aplicação de mecanismos de privacidade. As análises realizadas evidenciam a existência de um trade-off inerente entre privacidade e desempenho: quanto maiores as garantias de privacidade, maior é a degradação na acurácia e no *F1-score* dos modelos. Por outro lado, observa-se que arquiteturas mais robustas são capazes de mitigar parcialmente os efeitos adversos do ruído adicionado.

Os resultados obtidos reforçam a importância da Privacidade Diferencial como ferramenta essencial para a proteção de dados em ambientes de aprendizagem profunda, ao mesmo tempo em que destacam a necessidade de avanços metodológicos para reduzir o impacto negativo sobre o desempenho dos modelos.

Palavras-chave: Privacidade Diferencial, Aprendizagem Profunda, Redes Neurais, Proteção de Dados.

Abstract

Contemporary society is marked by the intense valuation and exploitation of data, driven by the rapid development of artificial intelligence technologies, particularly deep learning. Neural network-based models are capable of extracting complex patterns and enabling significant innovations across various areas of society. However, this scenario also highlights ethical and legal challenges concerning individual privacy, as many of these models are trained on large volumes of sensitive data.

Differential Privacy emerges as a fundamental theoretical and practical paradigm to mitigate the risks associated with personal information leakage. When applied to deep learning, it aims to ensure that the inclusion or removal of a single data point does not significantly affect the model's behavior, thereby protecting individual privacy even against adversaries with considerable inference capabilities.

This work investigates the application of Differential Privacy in the training of deep neural network models, employing the DP-SGD (Differentially Private Stochastic Gradient Descent) algorithm. Experiments were conducted using two architectures of varying complexities on the MNIST dataset, comparing model performance with and without the application of privacy mechanisms. The analyses reveal the inherent trade-off between privacy and performance: stronger privacy guarantees result in greater degradation of accuracy and *F1-score*. On the other hand, it was observed that more robust architectures can partially mitigate the adverse effects of the added noise.

The findings reinforce the importance of Differential Privacy as an essential tool for data protection in deep learning environments, while also underscoring the need for methodological advancements to reduce its negative impact on model performance.

Keywords: Differential Privacy, Deep Learning, Neural Networks, Data Protection.

Sumário

1 Introdução **1**

1.1 Motivação 1

1.2 Objetivos 3

1.2.1 Objetivos Gerais 4

1.2.2 Objetivos Secundários 4

1.3 Organização do texto 5

2 Conceitos Fundamentais **6**

2.1 Fundamentos da Aprendizagem de Máquina 6

2.1.1 Aprendizagem Profunda 7

2.1.2 Camadas Densas 8

2.1.3 Funções de Ativação 9

2.1.4 Gradiente Descendente 10

2.1.5 Dropout 11

2.1.6 Batch Normalization 11

2.1.7 Função Softmax 11

2.1.8 Redes Neurais Convolucionais 12

2.2 Fundamentos da Privacidade Diferencial 13

2.2.1 Garantias Oferecidas 13

2.2.2 Definição Formal 14

2.2.3 Sensibilidade da Função 14

2.2.4 Mecanismos de Privacidade 15

2.2.5 Modelos de Arquitetura 16

2.3 Ataques de Inferência 17

3 Revisão da Literatura **18**

3.1 Aprendizado com Gradiente Descendente Estocástico Diferencialmente Privado (*Differentially-Private Stochastic Gradient Descent* - DP-SGD) 18

3.2 Privacidade Diferencial via Treinamento Federado (*Differential Privacy Federated Federated Learning* - DP-FL) 19
3.3 Mecanismos Baseados em Redes Geradoras Adversariais(*Differentially Private Generative Adversarial Network* DP-GANs) 20
3.4 Privacidade Diferencial via Agregação e Pós-processamento 21

4 Metodologia **22**
4.1 Conjunto de Dados 22
4.2 Métricas de Avaliação de Desempenho 22
4.3 Técnicas Implementadas 23
4.4 Arquiteturas Utilizadas 25

5 Experimentos Computacionais **30**
5.1 Ambiente Computacional 30
5.2 Hiperparâmetros em PD 31
5.3 Configurações dos Experimentos 32
5.4 Análise de Desempenho e Privacidade 33
5.5 Tempo de Treinamento 35
5.6 Análise do Trade-off Privacidade vs Utilidade 36

6 Conclusão **38**
6.1 Considerações Finais 38
6.2 Trabalhos Futuros 39

Lista de Figuras

1 Arquitetura da AlexNet 7
2 Ilustração de camadas densas 8
3 Função de ativação RELU. 9
4 Modelos de PD. 16
5 Arquitetura A 26
6 Arquitetura B 28
7 Gráfico de acurácia para as arquiteturas contempladas, com e sem PD. . . . 33
8 Gráfico do tempo de treinamento para as arquiteturas consideradas, com e sem PD. 35
9 Gráfico de compromisso entre privacidade e utilidade. 36

1 Introdução

1.1 Motivação

O mundo se encontra na era da informação, um período em que dados digitais se tornaram um dos recursos mais valiosos da sociedade moderna, superando até mesmo o petróleo em importância estratégica [10]. Organizações, governos, e indivíduos produzem e consomem volumes massivos de dados diariamente, alimentando uma economia digital baseada no processamento e análise dessas informações. Estima-se que, em 2025, mais de 180 zettabytes de dados serão criados globalmente, um crescimento exponencial impulsionado por tecnologias como a Internet das Coisas (*Internet of Things* - IoT), redes sociais, computação em nuvem e inteligência artificial (IA) [17]. Esses dados, em grande parte, são de natureza pessoal e sensível — contendo hábitos de consumo, localização geográfica, histórico de saúde, preferências políticas e comportamentais, entre outros.

O valor dos dados, no entanto, está diretamente relacionado à sua sensibilidade. A capacidade de transformar dados pessoais em predições comportamentais e decisões automatizadas é o que impulsiona o sucesso de grandes empresas de tecnologia, como Google, Amazon, Meta e Alibaba [5, 20]. No entanto, esse mesmo poder levanta sérias preocupações éticas e sociais. À medida que sistemas baseados em aprendizado de máquina (AM) se tornam mais difundidos, cresce também o risco de que informações individuais sejam indevidamente acessadas, inferidas ou exploradas — mesmo quando os dados parecem ter sido anonimizados [23].

Casos notórios de vazamentos e abusos reforçam a gravidade da situação. O incidente com a empresa Equifax[1], em 2017, expôs informações financeiras e dados pessoais de mais de 140 milhões de cidadãos norte-americanos, resultando em consequências econômicas e legais significativas. Outro caso emblemático é o escândalo da Cambridge Analytica, que revelou como dados de usuários do Facebook foram coletados e utilizados, sem consentimento explícito, para influenciar processos eleitorais em diversas partes do mundo [16]. Tais eventos não apenas comprometem a privacidade de milhões de indivíduos, mas também minam a confiança pública em plataformas tecnológicas e instituições governamentais.

Diante desse cenário, cresce a demanda por mecanismos eficazes de proteção da privaci-

[1] Incidente da Equifax - acesso em 29/07/2025

dade. A simples anonimização ou remoção de identificadores diretos não é mais suficiente. Diversos estudos já demonstraram que indivíduos podem ser reidentificados por meio de correlações entre diferentes conjuntos de dados, mesmo após processos tradicionais de desidentificação [31]. A privacidade, nesse novo contexto, precisa ser tratada como uma garantia matemática, não como uma promessa técnica.

Nesse ponto, a privacidade diferencial (*Differential Privacy* - DP) surge como uma das abordagens mais robustas e rigorosamente definidas para o problema. Introduzida por Dwork *et al.* [7], DP define um modelo formal que garante que a presença ou ausência de qualquer indivíduo em um conjunto de dados não afeta substancialmente os resultados de qualquer análise estatística realizada. Em outras palavras, fornece uma garantia de que nenhuma informação específica sobre um indivíduo possa ser inferida com probabilidade significativa, mesmo com acesso ao resultado final da análise. Essa abordagem torna-se particularmente importante em sistemas que lidam com AM, nos quais conjuntos de dados são usados para treinar modelos complexos.

Ao proteger o modelo contra ataques de inferência, como ataques de inferência de associação e ataques de inversão de modelo [29], a privacidade diferencial permite que informações sensíveis sejam processadas com maior segurança. Isso é especialmente relevante em contextos como saúde, finanças, educação e segurança pública, nos quais a exposição de dados pode resultar em consequências graves, como discriminação, estigmatização ou fraudes [27].

Além disso, leis e regulamentações como o Regulamento Geral sobre a Proteção de Dados (GDPR)[2] da União Europeia, a Lei Geral de Proteção de Dados (LGPD)[3] no Brasil e a *California Consumer Privacy Act* (CCPA)[4] nos Estados Unidos têm impulsionado a adoção de tecnologias que assegurem o uso responsável dos dados pessoais. Tais normas impõem obrigações rigorosas às organizações, exigindo não apenas o consentimento explícito do usuário, mas também medidas técnicas e organizacionais que assegurem a confidencialidade, integridade e privacidade das informações [34].

No entanto, garantir tais níveis de proteção em ambientes de aprendizagem profunda (AP) representa um desafio técnico significativo. Modelos de redes neurais são notoriamente suscetíveis a memorizar dados específicos, principalmente quando expostos a grandes quantidades de exemplos durante o treinamento [30]. Isso compromete diretamente a privacidade

[2] Regulamento Geral sobre a Proteção de Dados (GDPR) - acesso em 29/07/2025
[3] Lei Geral de Proteção de Dados (LGPD) - acesso em 29/07/2025
[4] *California Consumer Privacy Act* (CCPA) - acesso em 29/07/2025

dos dados, ainda que o modelo final não contenha, explicitamente, nenhum identificador pessoal. Ferramentas como o algoritmo *Differentially Private Stochastic Gradient Descent* (DP-SGD) surgem como soluções práticas, ao incorporar ruído controlado no processo de treinamento e limitar a influência de qualquer exemplo individual sobre os parâmetros finais do modelo [1].

A crescente relevância da privacidade dos dados também coincide com o rápido avanço da AP, uma subárea da AM que tem alcançado resultados excepcionais em tarefas como reconhecimento de imagem, tradução automática, síntese de voz e análise de linguagem natural. Desde 2012, com a vitória da rede neural AlexNet na competição ImageNet, redes neurais profundas passaram a dominar diversos benchmarks, superando significativamente abordagens tradicionais. Essa revolução tecnológica impulsionou o uso de grandes volumes de dados para treinar modelos cada vez mais complexos, capazes de extrair padrões sofisticados e tomar decisões autônomas em contextos reais.

Contudo, a eficácia desses modelos está diretamente ligada à quantidade e à qualidade dos dados utilizados durante o treinamento [1] — frequentemente oriundos de informações sensíveis de indivíduos. Essa dependência por grandes conjuntos de dados aumenta os riscos à privacidade e evidencia a necessidade de mecanismos que permitam usufruir dos benefícios da AP sem comprometer a segurança ou a confidencialidade dos dados. É nesse cenário que a integração entre privacidade diferencial e AP se mostra particularmente promissora: ela permite proteger formalmente os dados dos usuários sem abrir mão da capacidade expressiva dos modelos neurais.

1.2 Objetivos

Diante da crescente preocupação com a privacidade dos dados em aplicações baseadas em inteligência artificial, torna-se fundamental investigar abordagens que permitam conciliar o uso de informações sensíveis com a proteção dos direitos dos indivíduos. Nesse contexto, este estudo visa explorar a aplicação da privacidade diferencial no domínio da AP, identificando seus benefícios, limitações e potencial prático. Os objetivos estão organizados em dois níveis: o objetivo geral, que define a meta principal do trabalho, e os objetivos específicos, que detalham as etapas necessárias para seu alcance.

1.2.1 Objetivos Gerais

O objetivo deste trabalho está inserido em um contexto global de crescente valorização dos dados e da privacidade individual. Avaliar o impacto da aplicação de técnicas de privacidade diferencial em redes neurais profundas não é apenas uma questão técnica, mas também uma resposta necessária à demanda ética e legal por práticas mais seguras de tratamento de dados. Este estudo visa contribuir com uma análise empírica que ajude a entender melhor o custo-benefício de tais abordagens, especialmente quanto à sua viabilidade prática e impacto na performance dos modelos de AP.

Busca-se entender até que ponto a privacidade diferencial pode ser efetivamente utilizada como uma ferramenta viável para proteger dados sensíveis sem comprometer significativamente o desempenho ou a utilidade dos modelos treinados para a tarefa de classificação de imagens.

Este trabalho contempla uma abordagem experimental para verificar os aspectos supracitados. Primeiramente, apresenta-se uma revisão dos conceitos essenciais de privacidade diferencial, com foco nos mecanismos utilizados em AM, como o DP-SGD. Em seguida, são exploradas implementações reais dessas técnicas em modelos de AP, com experimentos computacionais conduzidos em diferentes configurações arquiteturais. A avaliação considera métricas de desempenho como acurácia e *F1-score*, bem como o comportamento dos modelos sob diferentes níveis de privacidade (ϵ).

1.2.2 Objetivos Secundários

- Revisar os fundamentos teóricos da privacidade diferencial, com ênfase em sua aplicação no contexto de redes neurais profundas.
- Investigar as principais técnicas de implementação de privacidade diferencial em AP, com foco no algoritmo DP-SGD.
- Construir duas arquiteturas de redes neurais com diferentes níveis de complexidade para avaliar o impacto da aplicação da privacidade diferencial.
- Realizar experimentos computacionais comparando o desempenho de modelos treinados com e sem a aplicação de mecanismos de privacidade diferencial, utilizando métricas como acurácia e *F1-score*.

- Analisar os efeitos da variação do parâmetro de privacidade (ϵ) no desempenho dos modelos, identificando possíveis *trade-offs* entre privacidade e performance.
- Identificar oportunidades e lacunas para futuras pesquisas na área, com base nos resultados experimentais obtidos.

1.3 Organização do texto

Capítulo 2: Apresenta os conceitos básicos de AP e privacidade diferencial necessários para a compreensão deste trabalho;

Capítulo 3: Introduz e descreve as descobertas feitas por trabalhos relacionados;

Capítulo 4: Descreve a metodologia utilizada neste trabalho;

Capítulo 5: Expõe os resultados obtidos nos experimentos realizados;

Capítulo 6: Traz as conclusões finais e aponta possíveis caminhos de futuras pesquisas.

2 Conceitos Fundamentais

Este capítulo apresenta os conceitos fundamentais necessários para a compreensão deste trabalho. Inicialmente, são discutidos os princípios da Aprendizagem de Máquina e da Aprendizagem Profunda, que constituem a base técnica do que foi estudado. Em seguida, são abordados os fundamentos da Privacidade Diferencial, uma técnica central para a proteção de dados sensíveis em ambientes de AM.

2.1 Fundamentos da Aprendizagem de Máquina

A Aprendizagem de Máquina (*Machine Learning* - AM) é um subcampo da Inteligência Artificial que permite que sistemas computacionais aprendam padrões e realizem predições ou decisões com base em dados, sem serem explicitamente programados para isso. Segundo Tom Mitchell, "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."[22]. Em essência, algoritmos de AM contém modelos matemáticos que ajustam seus parâmetros a partir de dados históricos, utilizando esses modelos para generalizar sobre novos dados.

Os algoritmos de AM podem ser categorizados em quatro principais tipos: aprendizado supervisionado, aprendizado não supervisionado, aprendizado por reforço e aprendizado auto-supervisionado. No aprendizado supervisionado, o algoritmo recebe exemplos constituídos de pares de entradas e saída esperadas, aprendendo a mapear tais entradas nas respectivas saídas. No aprendizado não supervisionado, não há rótulos explícitos, e o algoritmo busca estruturas ou agrupamentos naturais nos dados, como por exemplo, algoritmos de clusterização ou de redução de dimensionalidade. Já no aprendizado por reforço, um agente aprende a tomar decisões com base em recompensas ou punições, por meio de interações com um ambiente, como por exemplo, um jogo. O aprendizado auto-supervisionado, por sua vez, representa uma abordagem intermediária, na qual o próprio sistema gera rótulos a partir dos dados brutos para realizar tarefas auxiliares, servindo como uma forma eficaz de pré-treinamento para modelos que posteriormente serão ajustados com dados rotulados.

Com o crescimento da disponibilidade de grandes volumes de dados e poder computacional, a AM passou a ser amplamente aplicada em tarefas como reconhecimento de voz,

diagnóstico médico, reconhecimento facial, sistemas de recomendação e detecção de objetos em imagens.

2.1.1 Aprendizagem Profunda

A Aprendizagem Profunda (*Deep Learning* - AP) é um subcampo da AM que utiliza redes neurais artificiais com múltiplas camadas para aprender representações complexas dos dados. A principal vantagem da AP está em sua capacidade de extrair automaticamente características relevantes dos dados brutos, sem a necessidade de engenharia manual de atributos (*featuring engineering*) [14].

As redes neurais profundas consistem em múltiplas camadas de unidades chamadas neurônios artificiais, organizadas em camadas de entrada, ocultas e de saída. Cada neurônio realiza uma transformação linear dos dados recebidos seguida de uma função de ativação não linear, a ser explicada mais adiante. A profundidade da rede — ou seja, o número de camadas ocultas — permite a aprendizagem de representações hierárquicas, em que camadas inferiores capturam características mais simples dos dados, enquanto camadas superiores capturam características mais abstratas.

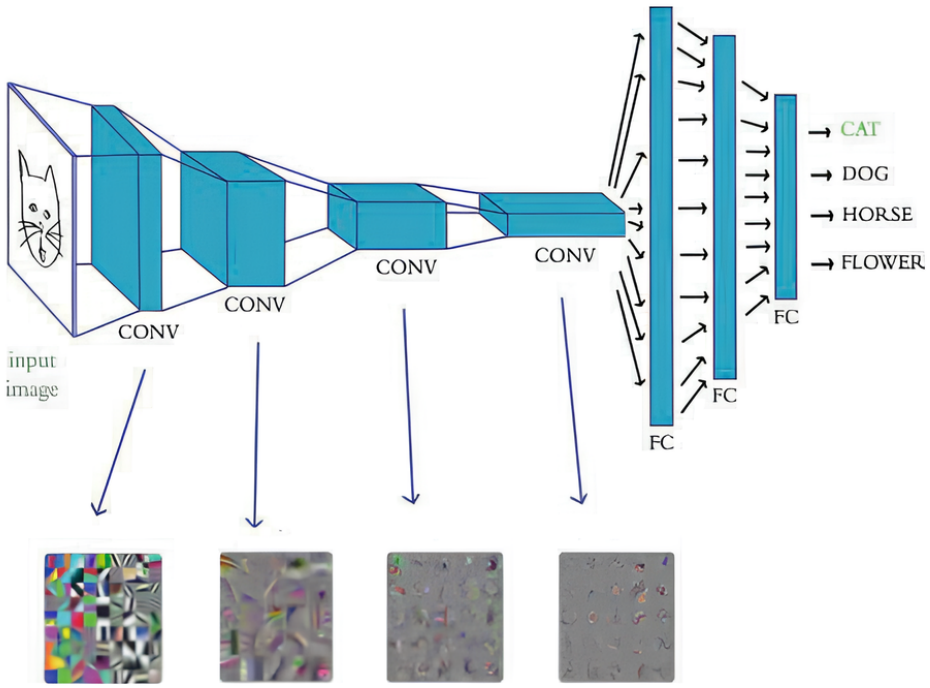

Figura 1: Arquitetura da AlexNet

Fonte: Extraida do livro "Deep Learning Illustrated: A Visual, Interactive Guide to Artificial Intelligence".

Dentre os modelos mais comuns de AP estão as *Convolutional Neural Networks* (CNNs), amplamente utilizadas em tarefas de visão computacional; as *Recurrent Neural Networks* (RNNs), aplicadas em dados sequenciais como texto e séries temporais; e os Transformadores, que se tornaram populares em tarefas de processamento de linguagem natural.

Apesar de seu grande potencial, os modelos de AP requerem grandes volumes de dados para treinamento eficaz [1]. Isso torna os sistemas baseados em AP especialmente suscetíveis a riscos relacionados à privacidade, uma vez que muitos desses dados podem ser de natureza pessoal ou sensível. É nesse ponto que técnicas como a Privacidade Diferencial se tornam relevantes, fornecendo uma estrutura matemática para mitigar tais riscos sem comprometer significativamente a utilidade dos modelos.

2.1.2 Camadas Densas

As camadas densas, também chamadas de *fully connected layers*, são um dos blocos mais fundamentais das redes neurais profundas. Nelas, cada neurônio da camada está conectado a todos os neurônios da camada anterior.

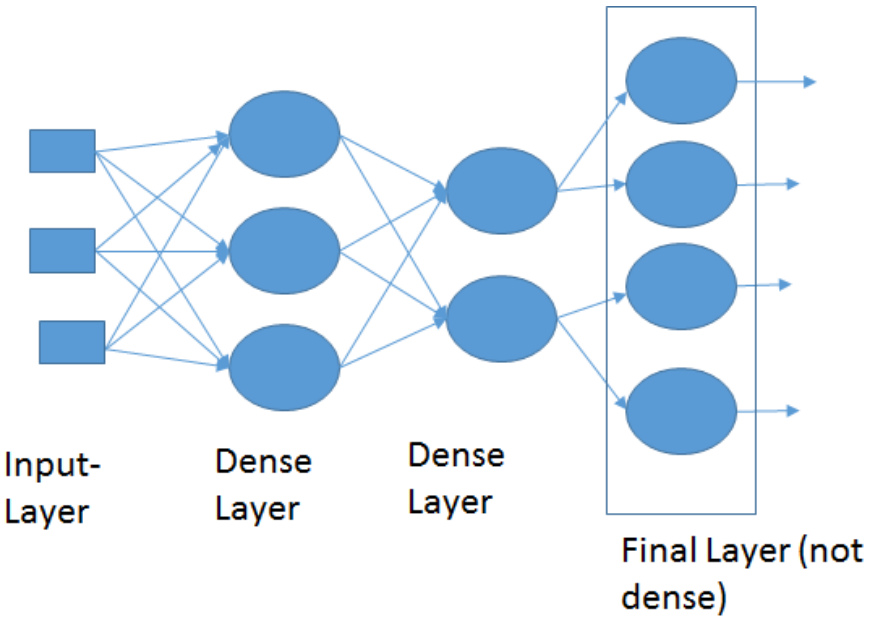

Figura 2: Ilustração de camadas densas

Fonte: https://stackoverflow.com/questions/63675602/how-to-implement-a-neural-network-with-a-not-fully-connected-layer-as-the-final - acesso em 29/07/2025.

Essa estrutura permite que o modelo aprenda relações complexas entre as entradas e saídas, sendo comumente utilizada nas etapas finais de redes para tarefas de classificação [2]. A

transformação realizada por uma camada densa pode ser matematicamente representada por uma multiplicação matriz-vetor seguida de uma função de ativação:

$$\mathbf{y} = \phi(\mathbf{W} \cdot \mathbf{x} + \mathbf{b}) \tag{1}$$

Onde $\mathbf{x}$ é o vetor de entrada, $\mathbf{W}$ é a matriz de pesos, $\mathbf{b}$ é o vetor de bias e ϕ representa a função de ativação.

2.1.3 Funções de Ativação

As funções de ativação desempenham um papel crucial ao introduzir não-linearidades na rede neural, permitindo que ela modele relações complexas nos dados. Entre as mais populares, destaca-se a função ReLU (*Rectified Linear Unit*), definida por:

$$f(x) = \max(0, x) \tag{2}$$

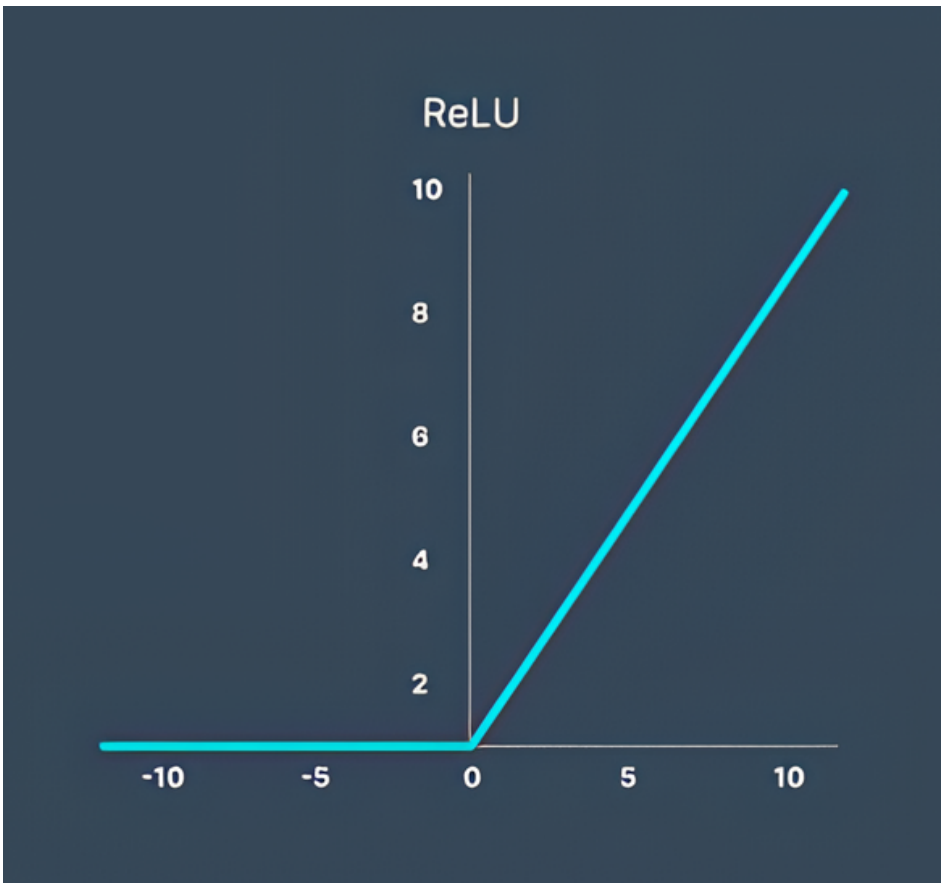

Figura 3: Função de ativação RELU.

Fonte: https://www.v7labs.com/blog/neural-networks-activation-functions - acesso em 29/07/2025.

A ReLU tornou-se a função padrão em muitas arquiteturas modernas devido à sua simplicidade computacional e capacidade de mitigar o problema do desaparecimento do gradiente (*vanishing gradient problem*) [13]. Outras funções de ativação, como Sigmoid e Tanh, também são utilizadas em contextos específicos, especialmente quando se deseja que a saída de um modelo corresponda a uma distribuição de probabilidade ou quando o problema requer ativações bem comportadas.

2.1.4 Gradiente Descendente

O treinamento de uma rede neural envolve a minimização de uma função de custo, que quantifica o erro entre as saídas previstas pela rede e as saídas reais. Para realizar o processo de minimização, é amplamente utilizado o algoritmo de Gradiente Descendente (*Gradient Descent*), que ajusta os pesos da rede iterativamente no sentido oposto da direção apontada pelo gradiente da função de custo em relação aos pesos do modelo [28].

Esse ajuste é feito de forma proporcional à contribuição de cada peso para o erro, permitindo que os pesos que mais influenciam negativamente o desempenho do modelo sejam corrigidos com maior intensidade.

A atualização dos pesos em uma iteração do Gradiente Descendente pode ser expressa como:

$$\theta = \theta - \eta g \tag{3}$$

Em que θ representa o vetor de pesos (parâmetros), η é a taxa de aprendizado (*learning rate*) e g é o gradiente da função de custo em relação aos pesos. O Gradiente Descendente é utilizado em conjunto com o algoritmo de Backpropagation, que aplica a regra da cadeia multivariada de forma eficiente e propaga o erro da camada de saída para as demais camadas anteriores.

Além da versão padrão, existem variantes como o Gradiente Descendente Estocástico (*Stochastic Gradient Descent* - SGD), que realiza as atualizações utilizando pequenos subconjuntos dos dados de treinamento (*minibatches*), e algoritmos como Adam e RMSProp, que introduzem melhorias em relação à forma base do SGD a fim de acelerar a convergência e evitar parar em ótimos locais de baixa qualidade.

Este conceito é especialmente relevante neste trabalho, pois as técnicas de privacidade diferencial para AP, como o DP-SGD, são diretamente baseadas na manipulação e na proteção

dos gradientes durante o processo de otimização. Conforme será apresentado no Capítulo 3 (Revisão da Literatura), o DP-SGD ajusta o fluxo de gradientes ao incorporar etapas de *clipping* e adição de ruído, alterando o comportamento tradicional do SGD para fornecer garantias formais de privacidade.

2.1.5 Dropout

O *dropout* é uma técnica de regularização amplamente utilizada em redes neurais profundas para reduzir o risco de *overfitting*. Durante o treinamento, ele consiste em desativar aleatoriamente uma fração dos neurônios de determinada camada em cada iteração, fazendo com que o modelo não dependa excessivamente de neurônios específicos para realizar previsões.

Esse processo força a rede a aprender representações mais robustas, uma vez que diferentes subconjuntos de neurônios são treinados em cada iteração. Na fase de inferência, todos os neurônios são ativados, mas seus pesos são ajustados para refletir a média das ativações utilizadas durante o treinamento. O *dropout* é simples de implementar e pode ser eficaz na melhoria da generalização do modelo [14].

2.1.6 Batch Normalization

A técnica de *batch normalization* foi introduzida para mitigar o problema de covariância interna (*internal covariate shift*), que ocorre quando a distribuição das ativações internas muda ao longo do treinamento. Essa mudança pode desacelerar a convergência e dificultar a escolha de taxas de aprendizado apropriadas.

A *batch normalization* atua normalizando a saída de cada camada para que possua média zero e variância unitária, com base nos valores do *batch* atual. Após essa normalização, são aplicados parâmetros aprendíveis de escala e deslocamento, permitindo que a rede recupere representações mais expressivas, se necessário. Essa técnica permite o uso de taxas de aprendizado maiores, acelera o treinamento e pode ter efeito regularizador, reduzindo a necessidade de outras técnicas como *dropout* em alguns casos [18].

2.1.7 Função Softmax

A função *softmax* é amplamente utilizada na camada de saída de redes neurais voltadas para tarefas de classificação multiclasse. Seu principal papel é transformar os valores produzidos pela rede em uma distribuição de probabilidade sobre as possíveis classes.

Essa transformação torna possível interpretar a saída da rede como a probabilidade estimada de cada classe ser a correta. A função garante que todos os valores resultantes estejam no intervalo entre 0 e 1, e que a soma total dessas probabilidades seja igual a 1. Dessa forma, a classe com maior valor na saída será aquela com maior probabilidade atribuída pelo modelo. Essa característica torna a *softmax* uma escolha natural para problemas em que se deseja prever uma única classe entre várias possibilidades[14].

2.1.8 Redes Neurais Convolucionais

As redes neurais convolucionais (*Convolutional Neural Networks* - CNNs) são arquiteturas especializadas para o processamento de dados com estrutura espacial ou temporal, como imagens e sinais. As CNNs utilizam filtros convolucionais que operam sobre regiões dos dados de entrada, extraindo características relevantes com menor número de parâmetros.

A operação de convolução é realizada por filtros - também chamados de *kernels* - que se deslocam sobre a entrada aplicando multiplicações ponto a ponto, produzindo mapas de ativação que realçam padrões específicos, como bordas ou texturas. Após as camadas convolucionais, é comum o uso de camadas de *pooling*, responsáveis por reduzir a dimensionalidade dos mapas de ativação, preservando as informações mais importantes e conferindo invariância espacial à rede. O tipo mais comum é o *max pooling*, que seleciona o valor máximo dentro de uma região específica da entrada.

Cada mapa de ativação gerado por um filtro convolucional é considerado um canal. À medida que os dados passam pelas camadas convolucionais, novos canais são criados. Cada filtro aprende a detectar um tipo específico de padrão, e sua saída se torna um novo canal. Assim, redes mais profundas acabam gerando dezenas ou até centenas de canais intermediários, cada um capturando diferentes aspectos da estrutura dos dados. Esses canais são empilhados e processados conjuntamente, permitindo que a rede represente informações complexas de forma distribuída. As imagens de entrada também possuem um ou mais canais. Em cenários em que a entrada do modelo são imagens coloridas, esses canais correspondem às cores (por exemplo, vermelho, verde e azul em imagens RGB) presentes na imagem.

A combinação dessas operações permite que as CNNs sejam altamente eficazes em tarefas de reconhecimento de padrões, como classificação de imagens e detecção de objetos, sendo amplamente utilizadas em sistemas modernos de visão computacional [2].

2.2 Fundamentos da Privacidade Diferencial

A privacidade diferencial (PD) foi introduzida por Cynthia Dwork *et al.* [7] como uma abordagem para fornecer garantias de privacidade em conjuntos de dados. A ideia central é garantir que a saída de um algoritmo de análise de dados não seja significativamente alterada pela inclusão ou exclusão de um único indivíduo, o que é matematicamente formalizado por um parâmetro de privacidade conhecido como ϵ (epsilon).

2.2.1 Garantias Oferecidas

As garantias oferecidas pela PD são matematicamente rigorosas [7]. Um de seus principais benefícios é a resistência a ataques de reidentificação, nos quais um invasor tenta inferir se os dados de um indivíduo específico estão presentes em um conjunto de informações, mesmo que esse conjunto tenha sido previamente anonimizado. Por exemplo, suponha que uma instituição de saúde publique estatísticas agregadas sobre pacientes, como distribuições de idade, diagnósticos ou tratamentos. Mesmo que os dados estejam anonimizados, um atacante com acesso a informações auxiliares — como a idade e o bairro de residência de certos indivíduos — poderia cruzar essas informações para tentar identificar pessoas específicas. No entanto, se as informações as quais o atacante tiver acesso forem geradas utilizando um mecanismo diferencialmente privado, a presença ou ausência de qualquer indivíduo terá impacto insignificante nos resultados finais. Isso impede que o atacante possa concluir com confiança se determinado indivíduo contribuiu para a base de dados, oferecendo proteção mesmo em cenários fora da inteligência artificial, como em censos, pesquisas de saúde pública ou análises estatísticas compartilhadas por órgãos governamentais.

Além disso, PD é composicional, o que significa que a aplicação de múltiplos algoritmos diferencialmente privados ao mesmo conjunto de dados ainda fornece garantias de privacidade quantificáveis. Neste contexto, cada operação — como uma consulta estatística, uma etapa de treinamento com DP-SGD ou uma análise de dados sensíveis — consome uma parte do orçamento total de privacidade. Este aspecto é tratado através dos teoremas de composição, que quantificam a perda acumulada de privacidade quando múltiplas operações são realizadas. Cynthia Dwork *et al.* [9] demonstraram que a composição de κ análises ϵ-diferencialmente privadas resulta em uma garantia de privacidade de até $\kappa\epsilon$.

2.2.2 Definição Formal

Formalmente, um algoritmo (ou mecanismo) $\mathcal{M}$ satisfaz (ϵ, δ)-privacidade diferencial se, para todos os conjuntos de dados D_1 e D_2 que diferem nos dados de um único indivíduo, e para todos os subconjuntos S do espaço de saída de $\mathcal{M}$:

$$Pr[\mathcal{M}(D_1) \in S] \leq e^{\epsilon} \cdot Pr[\mathcal{M}(D_2) \in S] + \delta \tag{4}$$

Aqui, ϵ (epsilon) é um parâmetro que controla o nível de privacidade: valores menores de ϵ fornecem maior privacidade. Este parâmetro representa o limite superior para a razão entre as probabilidades de uma determinada saída ser produzida por dois conjuntos de dados que diferem nos dados de um único indivíduo. Portanto, quanto menor o valor de ϵ, mais similar é a distribuição das saídas do algoritmo quando aplicado a D_1 e D_2, e maior a garantia de privacidade.

O parâmetro δ é uma probabilidade muito pequena de que o mecanismo viole essa garantia de forma significativa. Ou seja, com uma chance menor que δ, pode haver um vazamento maior do que o permitido por ϵ. Na prática, recomenda-se que δ seja escolhido como uma quantidade negligenciável em relação ao tamanho do conjunto de dados, como $\delta \leq \frac{1}{n}$ ou preferencialmente $\delta \leq \frac{1}{n^2}$, onde n é o número total de registros. Essa recomendação é destacada por Cynthia Dwork e Aaron Roth [8], que ressaltam a importância de manter δ suficientemente pequeno para que o risco de violações significativas permaneça praticamente inexistente em aplicações sensíveis.

2.2.3 Sensibilidade da Função

Outro conceito fundamental para a aplicação de PD é a sensibilidade da função. A sensibilidade, denotada como Δf, mede o quanto a saída de uma função f pode mudar com a adição ou remoção de um único elemento no conjunto de dados [7]. Formalmente, para uma função f que mapeia conjuntos de dados para valores reais, a sensibilidade é definida como:

$$\Delta f = \max_{D_1, D_2} \|f(D_1) - f(D_2)\|_p \tag{5}$$

Na equação acima, os conjuntos $\boldsymbol{D_1}$ e $\boldsymbol{D_2}$ são considerados vizinhos, ou seja, diferem em apenas um único elemento. A expressão $\|\boldsymbol{f}(\boldsymbol{D_1}) - \boldsymbol{f}(\boldsymbol{D_2})\|_{\boldsymbol{p}}$ representa a diferença entre as saídas da função $\boldsymbol{f}$ ao ser aplicada sobre esses dois conjuntos, medida segundo uma norma $\boldsymbol{p}$. A escolha dessa norma depende do mecanismo de privacidade diferencial adotado. Neste trabalho, a norma adotada para medir a sensibilidade foi a norma $\boldsymbol{\ell_2}$, pois foi utilizado o Mecanismo Gaussiano.

O valor de $\boldsymbol{\Delta f}$ corresponde, portanto, ao maior impacto que a adição ou remoção de um único registro pode causar na saída da função $\boldsymbol{f}$. Essa medida é crucial porque determina a quantidade de ruído que deve ser adicionada para garantir a PD. Funções com alta sensibilidade requerem mais ruído para manter o mesmo nível de privacidade.

No cenário de redes neurais, o objeto de interesse para aplicação da PD não é diretamente uma função escalar tradicional, mas sim os gradientes da função de custo, que orientam o processo de atualização dos parâmetros do modelo durante o treinamento. Em outras palavras, a "função" cuja sensibilidade precisa ser controlada é o cálculo dos gradientes individuais em cada iteração do algoritmo de otimização.

2.2.4 Mecanismos de Privacidade

Para garantir PD, diferentes mecanismos de perturbação são utilizados, os quais inserem ruído cuidadosamente calibrado com base na sensibilidade da função e no parâmetro ϵ. Os principais mecanismos utilizados são:

- **Mecanismo Laplaciano:** baseado na distribuição de Laplace, é utilizado quando a garantia de privacidade diferencial é puramente ϵ-DP. O ruído adicionado segue a distribuição Laplace$(0, \Delta f/\epsilon)$, onde Δf é a sensibilidade da função.

- **Mecanismo Gaussiano:** introduzido para satisfazer (ϵ, δ)-DP, adiciona ruído proveniente de uma distribuição Gaussiana (normal) com desvio padrão proporcional à sensibilidade e inversamente proporcional a ϵ. É particularmente útil quando se admite uma pequena probabilidade δ de falha nas garantias de privacidade.

- **Mecanismo Exponencial:** utilizado para funções com saída categórica ou discreta, seleciona uma saída de acordo com uma distribuição exponencial ponderada pela utilidade de cada saída, oferecendo uma maneira de aplicar PD sem necessariamente adicionar ruído contínuo.

2.2.5 Modelos de Arquitetura

Existem dois modelos de aplicação da PD, privacidade central e privacidade local.

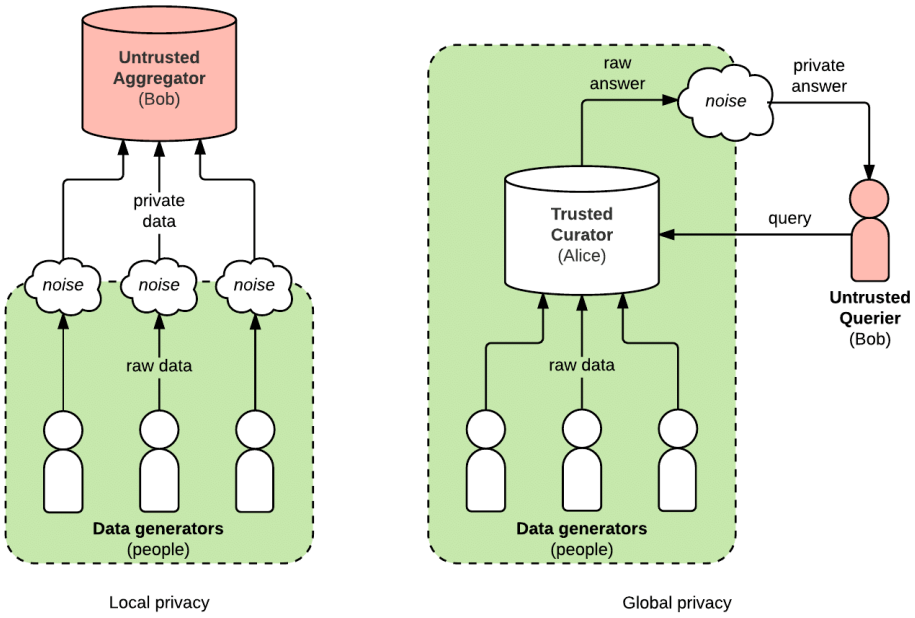

Figura 4: Modelos de PD.

Fonte: https://www.accessnow.org/understanding-differential-privacy-matters-digital-rights - acesso em 29/07/2025.

Em modelos de privacidade central - também chamado de privacidade global - , um servidor central de confiança tem acesso aos dados originais e aplica os mecanismos de PD antes de divulgar os resultados. Esse é o modelo mais comum em cenários como análise estatística de bases públicas ou treinamento de modelos de AM em instituições confiáveis.

Um cenário que exemplifica bem o uso de um modelo de privacidade central seria o de um instituto de pesquisa que coleta dados de pacientes para estudar padrões de saúde. Os dados são centralizados e o ruído é adicionado no momento da publicação de estatísticas ou modelos derivados. Desde que o instituto seja confiável, as garantias de PD são aplicadas de forma eficaz.

O modelo centralizado e o mais utilizado nos cenários de treinamento de redes neurais, tanto pela sua menor degradação de performance quanto pela sua maior facilidade de aplicação.

Nos modelos de privacidade local, cada usuário aplica o mecanismo de PD aos seus próprios dados antes de enviá-los ao servidor. Isso elimina a necessidade de confiar no servidor centralizado, tornando-o ideal para aplicações em que os usuários são relutantes em

compartilhar seus dados.

Um cenário de aplicação desse modelo seria, por exemplo, um aplicativo de teclado de *smartphone* que deseja coletar estatísticas sobre palavras mais usadas. Cada usuário adiciona ruído às suas informações localmente, enviando apenas versões perturbadas ao servidor. Essa abordagem foi popularizada pelo mecanismo RAPPOR do Google [11].

Ambas as abordagens possuem vantagens e limitações. A privacidade local oferece maior proteção contra servidores maliciosos, mas pode resultar em menor precisão devido à maior quantidade de ruído necessária. Já a privacidade central permite análises mais precisas, mas exige confiança na entidade central.

2.3 Ataques de Inferência

Ataques de inferência constituem uma classe de ameaças à privacidade em que um adversário tenta deduzir informações sensíveis a partir de dados acessíveis ou observações indiretas. Esses ataques não se limitam ao contexto de aprendizado de máquina, podendo ocorrer em bancos de dados estatísticos, sistemas de autenticação, ou qualquer ambiente em que respostas agregadas ou saídas computadas estejam disponíveis ao público.

No contexto específico de aprendizado de máquina, ataques de inferência se aproveitam da saída de modelos treinados para extrair dados privados usados durante o treinamento. Dentre os tipos mais discutidos, destacam-se os ataques de inferência de associação, nos quais o adversário tenta determinar se um dado específico participou do treinamento, e os ataques de inversão de modelo, que visam reconstruir entradas com base nas respostas do modelo [29]. Tais ameaças reforçam a importância do uso de técnicas como a privacidade diferencial para limitar o vazamento de informações sensíveis.

3 Revisão da Literatura

Esse capítulo discute as principais técnicas propostas na literatura para incorporar PD a algoritmos de AP, destacando os avanços, limitações e desafios enfrentados por essas abordagens.

3.1 Aprendizado com Gradiente Descendente Estocástico Diferencialmente Privado (*Differentially-Private Stochastic Gradient Descent* - DP-SGD)

Uma das contribuições mais influentes foi a introdução do algoritmo DP-SGD por Martin Abadi *et al.* [1]. Essa técnica adapta o tradicional método de SGD ao cenário de privacidade diferencial. O processo baseia-se na aplicação de duas operações principais: a limitação de gradiente (*gradient clipping*) e a adição de ruído gaussiano.

A limitação de gradiente restringe a influência de cada amostra (exemplo) individual sobre o gradiente total, reduzindo a sensibilidade da função de custo. Após essa etapa, adiciona-se ruído gaussiano calibrado com base em um parâmetro conhecido como multiplicador de ruído, na probabilidade de amostragem de cada exemplo por *minibatch* e no número total de atualizações dos pesos realizadas durante o processo de treinamento. Com isso, é possível controlar matematicamente a probabilidade de que um modelo revele a participação de uma amostra específica.

No contexto do DP-SGD, o ruído corresponde a valores aleatórios amostrados de uma distribuição estatística (normalmente uma distribuição Gaussiana de média zero e variância calibrada pelo multiplicador de ruído e pelo parâmetro de limitação de gradiente). Esse ruído é somado ao gradiente agregado após a etapa de limitação, de modo a mascarar a contribuição individual de cada exemplo presente no lote de treinamento. Em outras palavras, o ruído atua como uma perturbação controlada, garantindo que a saída do processo de treinamento não revele de forma precisa a presença ou ausência de qualquer amostra específica no conjunto de dados.

O orçamento de privacidade (ε, δ) em algoritmos como o DP-SGD é influenciado principalmente por três fatores: o multiplicador de ruído utilizado, a probabilidade de amostragem e o número total de atualizações de gradiente realizadas durante o treinamento.

A cada atualização, o modelo sofre uma pequena perda de privacidade, e a perda acu-

mulada ao longo de todo o treinamento é monitorada por mecanismos chamados de *privacy accountants*. O parâmetro δ, por sua vez, representa uma pequena probabilidade de falha nas garantias de privacidade, e é fixado como uma constante muito pequena (geralmente menor que o inverso do número de exemplos). Juntos, ε e δ quantificam de forma matemática o grau de proteção oferecido pelo mecanismo diferencialmente privado.

Vale ressaltar que o nível de ruído introduzido no gradiente não deve ser interpretado de forma isolada como equivalente ao nível de privacidade alcançado. Embora o ruído seja um dos elementos centrais do mecanismo, o grau de PD obtido depende de sua interação com os demais hiperparâmetros e com o número total de iterações de treinamento. Em termos práticos, isso significa que adicionar mais ruído não implica, necessariamente, em maior privacidade se o restante do processo não for considerado em conjunto. Assim, o valor de ϵ reportado representa a medida formal do nível de privacidade alcançado, e não o ruído em si.

Além disso, Martin Abadi *et al.* [1] introduziram o conceito de *momentum accounting*, uma técnica para acompanhar o gasto acumulado de privacidade ao longo das iterações. Essa abordagem melhorou significativamente a gestão da composição de mecanismos privados, possibilitando maior controle sobre o orçamento total de privacidade.

Diversos trabalhos posteriores refinaram e estenderam o DP-SGD. Por exemplo, Nicolas Papernot *et al.* [25] exploraram variantes do otimizador que incorporam estratégias como decaimento da taxa de aprendizado adaptativo e limitação de gradiente dinâmico, visando reduzir a degradação de desempenho causada pelo ruído adicionado. Ainda assim, uma limitação comum é o impacto negativo sobre a acurácia do modelo, especialmente em domínios com dados escassos ou desbalanceados.

Além da aplicação direta de ruído aos gradientes como no DP-SGD, outras propostas buscam introduzir mecanismos de PD em componentes internos das redes neurais. NhatHai Phan *et al.* [26], por exemplo, investigaram a adição de ruído laplaciano às ativações ReLU durante o treinamento, como forma de preservar a privacidade sem comprometer significativamente a performance do modelo.

3.2 Privacidade Diferencial via Treinamento Federado (*Differential Privacy Federated Federated Learning* - DP-FL)

O treinamento federado (*Federated Learning* - FL), introduzido por Brendan McMahan *et*

al. [21], permite, em um ambiente distribuído, a construção de modelos globais sem a necessidade de centralizar os dados dos usuários. Em vez disso, o modelo é compartilhado com os usuários que então realizam o treinamento localmente em seus dispositivos utilizando seus dados locais. Apenas os gradientes ou atualizações dos pesos obtidos no processo de treinamento são compartilhados com o servidor central. A integração de privacidade diferencial ao FL resulta em uma abordagem chamada DP-FL, altamente relevante em cenários contendo dispositivos móveis e *Internet of Things* (IoT).

Nessa configuração, o ruído é adicionado às atualizações obtidas no processo de treinamento local antes de sua agregação ao servidor. Uma das principais propostas nesse sentido é o modelo *Federated Averaging* com DP, em que o ruído gaussiano é adicionado após a aplicação de limitação às atualizações locais. Trabalhos como o de Robin Geyer *et al.* [12] e Stacey Truex *et al.* [33] demonstraram que essa abordagem é capaz de levar a modelos com bons níveis de desempenho mesmo sob restrições de privacidade estritas.

Um desafio importante do DP-FL está na gestão do ruído acumulado ao longo de várias rodadas de comunicação, principalmente em ambientes com conectividade instável ou dispositivos heterogêneos. Isso tem motivado a pesquisa por mecanismos adaptativos, nos quais a quantidade de ruído pode ser calibrada dinamicamente com base na qualidade das atualizações locais.

3.3 Mecanismos Baseados em Redes Geradoras Adversariais(*Differentially Private Generative Adversarial Network* DP-GANs)

Uma abordagem alternativa para preservar a privacidade em modelos de AP consiste na geração de dados sintéticos que sejam semelhantes aos dados reais, mas que não revelem informações específicas de indivíduos. As Redes Geradoras Adversariais (*Generative Adversarial Network* - GANs), introduzidas por Ian Goodfellow *et al.* [15], foram adaptadas para este fim com mecanismos de privacidade diferencial, resultando em modelos conhecidos como DP-GANs.

O trabalho de Liyang Xie *et al.* [35] propôs o DP-GAN, onde o discriminador da GAN é treinado com privacidade diferencial via ruído adicionado à sua função de perda. O objetivo é que o gerador aprenda a produzir amostras sintéticas estatisticamente similares aos dados reais, sem comprometer a privacidade dos indivíduos presentes no conjunto original.

Além disso, Reihaneh Torkzadehmahani *et al.* [32] exploraram variantes condicionais da

arquitetura GAN, possibilitando a geração de dados sintéticos com rótulos específicos, o que amplia sua aplicabilidade a problemas de classificação supervisionada. No entanto, essas abordagens enfrentam desafios técnicos como instabilidade no treinamento e dificuldade em preservar a utilidade estatística dos dados gerados.

3.4 Privacidade Diferencial via Agregação e Pós-processamento

Outra linha de pesquisa relevante consiste em aplicar mecanismos de PD não durante o treinamento, mas após a coleta ou agregação de dados. Isso inclui técnicas de *pós-processamento* que garantem privacidade na fase de análise, como a liberação de estatísticas com ruído calibrado.

Por exemplo, Jun Zhang *et al.* [37] exploraram o uso de privacidade diferencial em inferência bayesiana posteriori, onde os parâmetros do modelo são perturbados após o treinamento. Essa abordagem permite preservar a acurácia do modelo original, enquanto garante que a liberação pública do modelo final não comprometa a privacidade dos dados usados para treiná-lo.

Outro exemplo é o mecanismo de *aggregated teacher ensembles* proposto por Nicolas Papernot *et al.* [24], que utiliza múltiplos modelos treinados em subconjuntos disjuntos dos dados. As previsões desses modelos (*teachers*) são então agregadas com ruído antes de serem utilizadas para treinar um modelo final (*student*). O ruído adicionado à agregação das respostas dos modelos *teachers* garante que nenhuma informação individual possa ser inferida diretamente.

4 Metodologia

Para avaliar a eficácia das técnicas implementadas, foram realizados experimentos computacionais utilizando conjuntos de dados reais. Os experimentos foram projetados para avaliar, segundo métricas estabelecidas na literatura, os modelos treinados com PD e compará-los com modelos tradicionais sem garantias de privacidade.

O algoritmo de Descida de Gradiente Estocástico com Privacidade Diferencial (DP-SGD) foi implementado utilizando a biblioteca [5].

4.1 Conjunto de Dados

O conjunto de dados utilizado nos experimentos foi o MNIST [19], um conjunto de dados composto por imagens de dígitos manuscritos amplamente utilizado para treinar e testar modelos de classificação de imagens. Os principais motivos para a escolha desse conjunto de dados são o baixo custo computacional para o treinamento de uma rede com esses dados; visto que os recursos computacionais que se tem à disposição são limitados e o tempo de treinamento de modelos que implementam PD tendem a ser, em geral, elevados; e a grande quantidade de trabalhos já realizados com esse conjunto de dados, o que o torna uma boa base para comparação de desempenho.

4.2 Métricas de Avaliação de Desempenho

Para avaliar o desempenho dos modelos de classificação, foram utilizadas amplamente reconhecidas na literatura: acurácia, precisão, revocação (ou *recall*) e *F1-score*. Cada uma dessas métricas oferece uma perspectiva complementar sobre a qualidade das predições realizadas pelo modelo.

$$\text{Acurácia} = \frac{VP + VN}{VP + VN + FP + FN} \tag{6}$$

A acurácia representa a proporção de previsões corretas em relação ao total de exemplos

[5] Opacus - acesso em 29/07/2025

avaliados. Embora útil como medida geral, sua interpretação pode ser enganosa em cenários com classes desbalanceadas, nos quais a classe majoritária domina a métrica.

$$\text{Precisão} = \frac{VP}{VP + FP} \tag{7}$$

A precisão avalia a proporção de exemplos que foram classificados como positivos e que de fato pertencem à classe positiva. Essa métrica é particularmente importante em contextos onde os falsos positivos devem ser minimizados, como em diagnósticos médicos ou detecção de fraudes.

$$\text{Revocação} = \frac{VP}{VP + FN} \tag{8}$$

A revocação mede a capacidade do modelo de identificar corretamente todos os exemplos positivos. Ou seja, quantos dos elementos realmente positivos foram corretamente classificados como tal. Essa métrica é essencial em cenários onde perder exemplos positivos tem consequências críticas.

$$\text{F1-score} = 2 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}} \tag{9}$$

Por fim, o *F1-score* corresponde à média harmônica entre precisão e revocação, oferecendo uma medida equilibrada entre essas duas métricas. Ele é especialmente útil quando há um desequilíbrio entre as classes ou quando é necessário ponderar tanto os falsos positivos quanto os falsos negativos.

Os níveis de privacidade alcançados serão avaliados com base nos parâmetros ϵ obtidos ao final dos treinamentos. Os resultados dos experimentos foram analisados e discutidos para avaliar o impacto da PD no desempenho dos modelos de AP.

4.3 Técnicas Implementadas

A implementação seguiu os passos descritos por Martin Abadi *et al.* [1], baseando-se no uso

do algoritmo DP-SGD. O treinamento é realizado por meio de *minibatches* — subconjuntos do conjunto de dados selecionados em cada iteração — que são posteriormente divididos em unidades menores chamadas de *microbatches*. Cada *microbatch* contém uma fração dos exemplos do *minibatch* e é processado individualmente no cálculo do gradiente.

Para cada *microbatch*, os gradientes são computados separadamente e, em seguida, passam por uma operação chamada de *clipping*, que consiste na limitação de sua influência nos pesos a um valor máximo pré-definido. Essa etapa é fundamental para evitar que qualquer dado individual exerça influência excessiva sobre a atualização dos pesos. Após serem limitados, os gradientes de todos os *microbatches* pertencentes ao *minibatch* são somados, e ruído gaussiano é adicionado ao gradiente agregado. Esse ruído é calibrado conforme o parâmetro de multiplicador de ruído, afetando diretamente o valor de ϵ obtido e assegurando que a atualização dos pesos satisfaça os critérios formais da PD. Por fim, os pesos do modelo são atualizados com base no gradiente resultante perturbado.

Para tornar mais claro como o processo de treinamento com PD é conduzido, o pseudocódigo contido no Algoritmo 1 apresenta os passos do algoritmo DP-SGD, que foi previamente explicado.

Algorithm 1 DP-SGD: Descida de Gradiente Estocástico com Privacidade Diferencial

Require: Conjunto de dados de treinamento D, taxa de aprendizado η, número de épocas E, tamanho do lote L, número de *microbatches* M, parâmetro de limitação de gradiente C, e multiplicador de ruído σ

1: Inicializar os pesos do modelo θ de forma aleatória
2: **for** cada época $e = 1$ até E **do**
3: **for** cada *minibatch* B extraído de D (usando Poisson Sampling) **do**
4: **for** cada *microbatch* m em B **do**
5: Calcular o gradiente g_m da função de perda em relação a θ
6: Aplicar **limitação de gradiente com norma L2** ao gradiente: $g_m \leftarrow g_m \cdot \min\left(1, \frac{C}{\|g_m\|_2}\right)$
7: **end for**
8: Agregar os gradientes limitados: $g \leftarrow \frac{1}{M}\sum_{m=1}^{M} g_m$
9: Adicionar ruído gaussiano ao gradiente: $\tilde{g} \leftarrow g + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$
10: Atualizar os pesos do modelo: $\theta \leftarrow \theta - \eta\tilde{g}$
11: **end for**
12: **end for**

É importante notar que o algoritmo DP-SGD mantém a mesma estrutura básica do algoritmo tradicional de descida de gradiente estocástico (SGD), diferenciando-se em apenas duas etapas cruciais. Na linha 6, ocorre a limitação dos gradientes individuais por meio da norma L2, garantindo que a contribuição de cada exemplo de treino seja controlada. Já na

linha 9, adiciona-se ruído gaussiano ao gradiente agregado, assegurando as garantias formais de PD. Essas duas modificações, embora sutis em termos de implementação, são responsáveis por transformar o SGD convencional em um mecanismo compatível com os princípios de PD, sem alterar o restante da lógica de treinamento da rede neural.

Para garantir que as propriedades formais da PD sejam preservadas, foi utilizada uma estratégia de amostragem baseada na distribuição de Poisson, na qual cada exemplo do conjunto de dados é incluído no *minibatch* com uma probabilidade fixa $\boldsymbol{q}$, independentemente dos demais. Como essa abordagem resulta em *minibatches* de tamanho variável, foi realizado um ajuste posterior nos *minibatches* para garantir compatibilidade com as exigências do método de treinamento. Sempre que necessário, os *minibatches* tiveram seu tamanho adaptado por meio de truncamento ou preenchimento (*padding*) com exemplos zerados. Essas amostras garantem que todos os *minibatches* tenham dimensões consistentes, requisito técnico para otimização eficiente e correta aplicação dos mecanismos de privacidade. Além disso, por serem amostras artificiais compostas inteiramente por zeros, não carregam nenhuma informação sensível e não afetam negativamente as garantias formais de PD.

4.4 Arquiteturas Utilizadas

Nos experimentos computacionais conduzidos, foram treinadas duas redes neurais com arquiteturas distintas. Essas arquiteturas foram selecionadas com o objetivo de avaliar o impacto da aplicação de PD em cenários com diferentes níveis de complexidade estrutural. A Arquitetura A, mais simples e de menor profundidade, foi utilizada como base para observar de forma clara os efeitos da adição de ruído e da limitação de gradientes sobre modelos menos robustos. Por outro lado, a Arquitetura B incorpora técnicas de regularização, como *dropout* e *group normalization*, além de maior profundidade. Dessa forma, a comparação entre as duas arquiteturas possibilita avaliar não apenas o impacto direto da PD no treinamento, mas também como a complexidade e a presença de mecanismos de regularização podem mitigar a degradação de desempenho associada a esse contexto.

A primeira rede (A) possui uma arquitetura extremamente simples. Ela possui 2 blocos *convpool*, uma camada *flatten* e duas camadas densas; conforme exibido na figura 5. Essa arquitetura foi retirada da documentação do TensorFlow Privacy [6], em que é utilizada para fazer uma introdução a PD.

[6] Documentação do TensorFlow Privacy - acesso em 29/07/2025

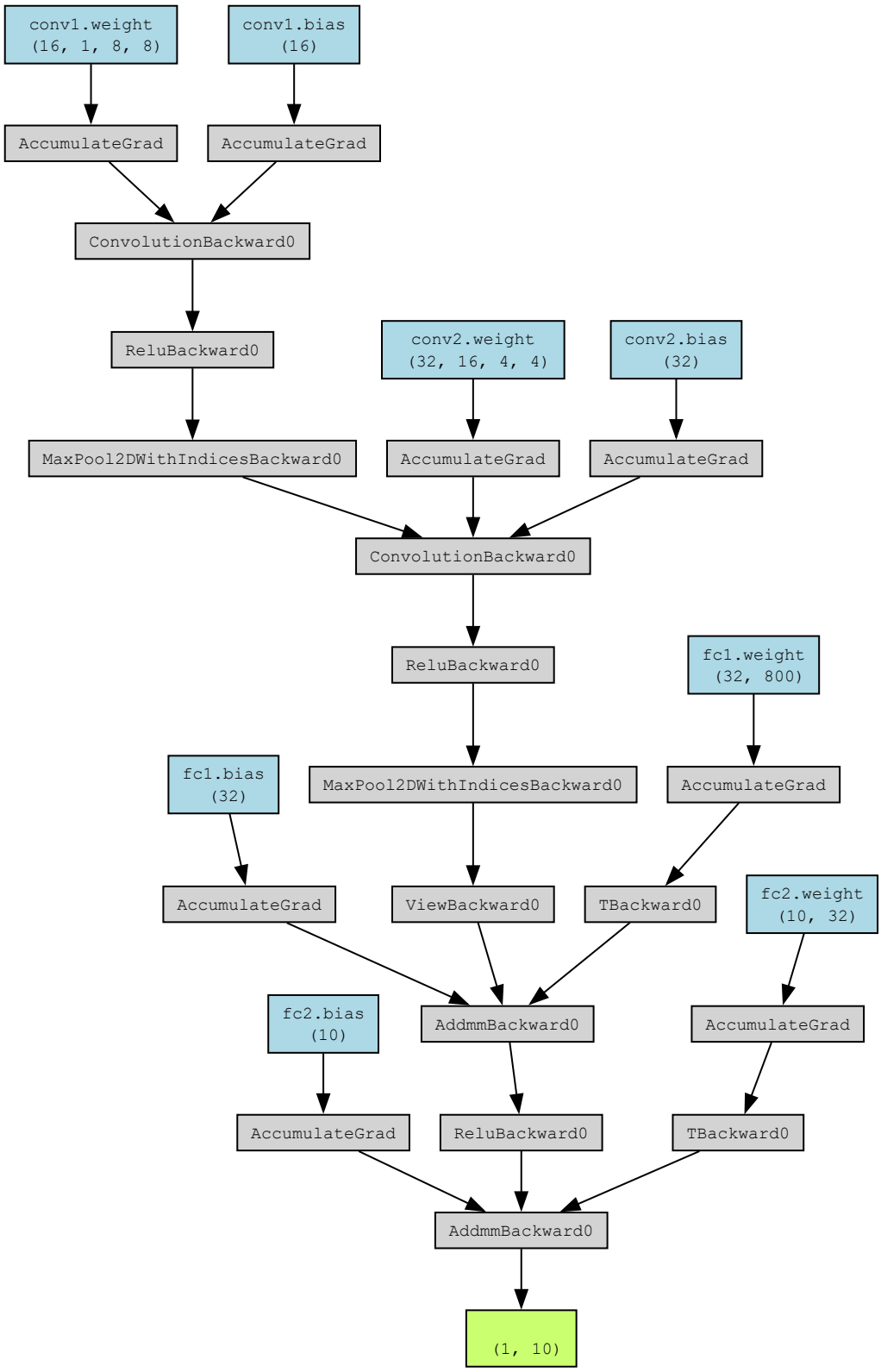

Figura 5: Arquitetura A

Fonte: Elaboração própria utilizando a biblioteca torchviz.

A segunda rede (B) possui uma arquitetura mais complexa em relação à primeira. Ela incorpora técnicas como *dropout* e *batch normalization*, além de utilizar a função *softmax* em sua camada de saída; conforme exibido na figura 6. Ela foi originalmente proposta por Chris Deotte [6], em uma competição sobre o dataset MNIST, e garantiu a sua vitória na competição. Essa arquitetura se encontra em um repositório público no GitHub[7].

No entanto, considerando que o cenário de treinamento deste trabalho envolve a aplicação de PD, todas as camadas de *batch normalization* foram substituídas por *group normalization*. Essa decisão foi tomada porque a *batch normalization* depende da média e variância calculadas sobre todos os exemplos do *minibatch*, o que é problemático em cenários com PD.

Por outro lado, a camada de *group normalization* realiza a normalização com base em grupos de canais dentro de cada exemplo individual, ao invés de depender das estatísticas globais do *minibatch*. Isso a torna mais adequada para cenários com tamanhos de *minibatch* pequenos ou variáveis, tal como os cenários de treinamento realizados neste trabalho, em que existe aplicação de PD e amostragem da distribuição Poisson.

[7] Repositório Chris Deotte - acesso em 29/07/2025

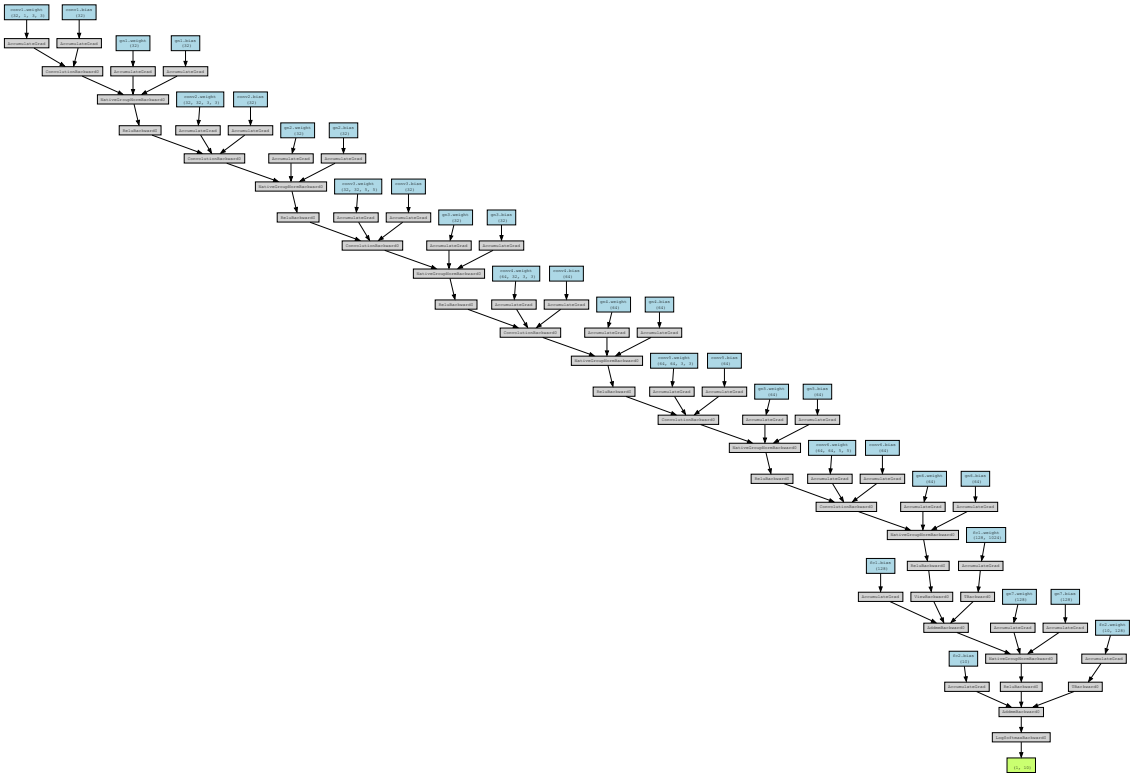

Figura 6: Arquitetura B

Fonte: Elaboração própria utilizando a biblioteca torchviz.

Para cada arquitetura, foram testadas configurações com e sem aplicação de PD. A técnica utilizada para introduzir PD foi o algoritmo DP-SGD, com hiperparâmetros como taxa de aprendizagem, tamanho de *batch*, multiplicador de ruído, limitação do gradiente e uso de *microbatches* sendo ajustadas entre os experimentos. As métricas de avaliação analisadas incluíram acurácia, precisão, *recall*, *F1-score*, tempo de treinamento e a garantia de privacidade obtida.

5 Experimentos Computacionais

Este capítulo apresenta o ambiente computacional, os cenários adotados nos experimentos, as variações de hiperparâmetros testadas, os resultados obtidos e uma análise interpretativa desses dados. O objetivo central é avaliar os impactos da aplicação da PD no desempenho de modelos de AP, comparando arquiteturas distintas, com e sem a aplicação de mecanismos de PD.

5.1 Ambiente Computacional

Este trabalho foi desenvolvido em um contexto em que não se dispunha de recursos computacionais poderosos tais como aqueles adotados por grandes empresas que realizam pesquisa em AP. Dessa forma, os cenários pensados envolvem menor demanda computacional para serem executados. O seguinte ambiente computacional foi utilizado nos experimentos:

Processador: Intel(R) Core(TM) i5-10400F CPU @ 2.90GHz.
Memória: 16,0 GB de memória RAM DDR4.
Placa de Vídeo: NVIDIA GeForce RTX 3060 Ti.
Sistema Operacional: Windows 10 Pro, versão 22H2.

O desenvolvimento e a execução dos experimentos deste trabalho foram realizados utilizando Python na versão 3.10, em conjunto com bibliotecas especializadas para desenvolvimento de modelos de AP, análise de dados e privacidade diferencial. A seguir, são descritas as bibliotecas e *frameworks* utilizados:

- **PyTorch**[8]: *Framework* principal de AP utilizado para a construção e o treinamento das redes neurais. Oferece uma interface flexível para definição de modelos, manipulação de tensores e execução de operações computacionais intensivas em GPU (Graphic Processing Unit).
- **Torchvision**: Biblioteca complementar ao PyTorch, utilizada para acesso a conjuntos

[8] PyTorch - acesso em 29/07/2025

de dados públicos, como o MNIST, além de fornecer ferramentas para transformações e pré-processamento de imagens.

- **Opacus**[9]: Biblioteca desenvolvida pela Meta AI, especializada em fornecer suporte para a implementação de PD em modelos de AP treinados com PyTorch. O Opacus permite a aplicação eficiente do algoritmo DP-SGD, oferecendo recursos nativos para realizar limitação do gradiente, amostragem por distribuição de Poisson e adição automática de ruído gaussiano aos gradientes durante o treinamento. Além disso, a biblioteca calcula e monitora o orçamento de privacidade acumulado (ϵ) ao longo das iterações, facilitando o controle sobre os níveis de privacidade durante os experimentos.

- **Scikit-learn**[10]: Biblioteca amplamente utilizada para análise de dados e aprendizado de máquina. Foi empregada para o cálculo de métricas de avaliação dos modelos, incluindo Acurácia, Precisão, Revocação e *F1-score*.

- **itertools**: Biblioteca padrão da linguagem Python, utilizada para a geração eficiente de combinações de hiperparâmetros durante os experimentos.

- **time**: Também pertencente à biblioteca padrão do Python, foi utilizada para medir o tempo total de execução de cada experimento.

- **tqdm**: Biblioteca utilizada para exibir barras de progresso durante os ciclos de treinamento, proporcionando melhor acompanhamento visual da execução dos experimentos.

O Pytorch foi utilizado na versão 2.7.0 com CUDA 12.8. Todo o código desenvolvido e resultados obtidos estão disponíveis em um repositório público no Github[11]. Nesse repositório também existe uma implementação utilizando a versão 2.15 do tensorflow.

5.2 Hiperparâmetros em PD

A definição de hiperparâmetros em modelos treinados com PD é um dos principais desafios relatados na literatura. Ainda não existe um consenso estabelecido sobre quais valores devem ser utilizados para parâmetros como limitação de gradiente e multiplicador de ruído. A

[9] Opacus - acesso em 29/07/2025
[10] Scikit-learn - acesso em 29/07/2025
[11] Repositório deste trabalho - acesso em 29/07/2025

escolha adequada depende fortemente do contexto de aplicação, da complexidade dos dados, do tamanho do conjunto de treinamento e até mesmo da arquitetura utilizada. A determinação desses valores exige experimentação cuidadosa, com ajustes empíricos que busquem equilibrar a privacidade garantida e a utilidade do modelo final [1, 8, 3].

Na prática, a literatura costuma relatar valores de ϵ variando entre 1 e 10 sendo associados a níveis considerados razoáveis de privacidade em muitos cenários. δ é geralmente escolhido como um valor muito pequeno, tipicamente da ordem de $1/N$ (sendo N o número de exemplos do conjunto de dados). Já o multiplicador de ruído e a limitação de gradiente são parâmetros ainda mais sensíveis, que devem ser ajustados em função da escala dos gradientes e da robustez da arquitetura empregada. Por essa razão, não há recomendações universais, apenas faixas exploratórias relatadas em diferentes experimentos [4].

Nos experimentos realizados neste trabalho, adotou-se $\delta = 10^{-5}$, valor em linha com boas práticas descritas na literatura. O ϵ foi calculado através do método de *Moments Accountant*. A limitação de gradiente e o multiplicador de ruído foram configurados como hiperparâmetros ajustáveis, variando em diferentes cenários para avaliar a degradação de desempenho em função da privacidade. Os testes realizados possuem um caráter exploratório, com o objetivo de encontrar valores que melhorem a garantia de privacidade do modelo sem uma alta degradação no desempenho.

5.3 Configurações dos Experimentos

Os treinamentos foram conduzidos considerando cenários com e sem PD. No cenário sem PD, foi utilizado como otimizador o SGD tradicional. Já no cenário com PD, utilizou-se o algoritmo DP-SGD, conforme descrito por Martin Abadi *et al.* [1], com suporte da biblioteca Opacus, que aplica limitação de gradiente e ruído gaussiano aos gradientes de acordo com o orçamento de privacidade definido. Também foi utilizada amostragem de Poisson para seleção dos exemplos que iriam compor o *batch*, conforme recomendação da própria biblioteca.

Os testes variaram os seguintes hiperparâmetros:

- **taxa de aprendizagem:** [0.01, 0.005, 0.001]
- **tamanho do *batch*:** [25, 50, 100]
- **valor de limitação de gradiente:** [0.5, 1.0, 1.5]

- **multiplicador de ruído:** [0.5, 1.0, 1.5]
- **número de *microbatches*:** [5, 10]

Cada combinação foi executada em ambas as arquiteturas, com e sem a aplicação de PD. Resultando no treinamento de 324 modelos com diferentes configurações para cada uma das duas arquiteturas (648 modelos no total).

5.4 Análise de Desempenho e Privacidade

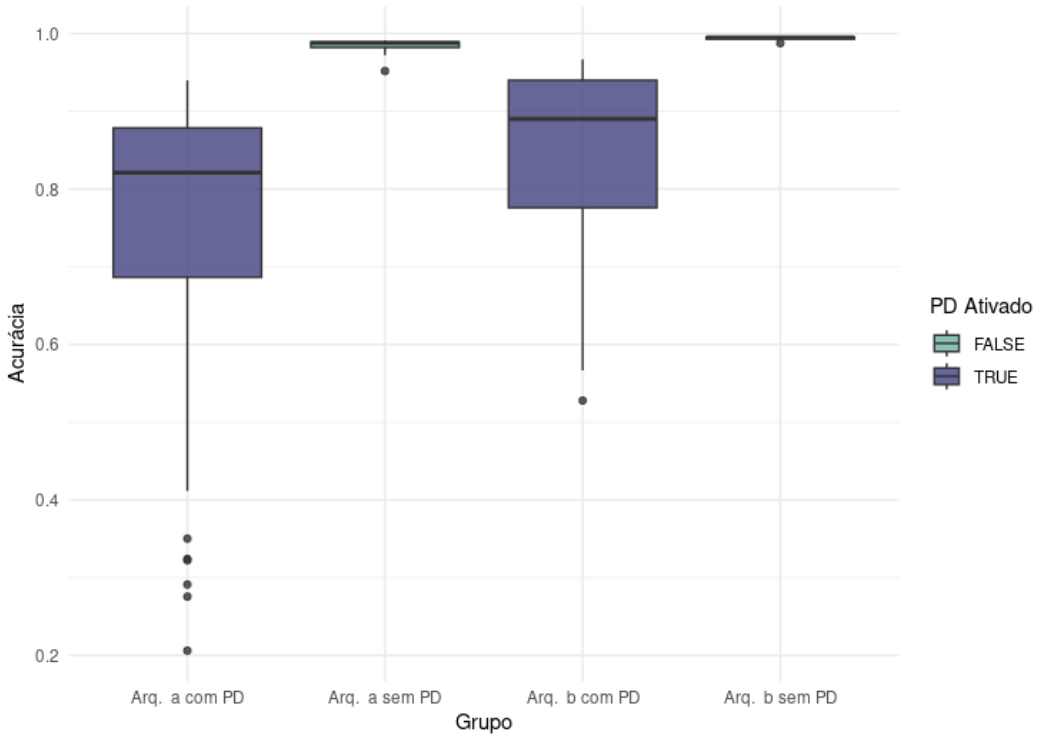

Figura 7: Gráfico de acurácia para as arquiteturas contempladas, com e sem PD.

Fonte: Elaboração própria.

Os resultados demonstram, de forma clara, o impacto negativo da aplicação de PD na acurácia dos modelos. A média de acurácia dos modelos sem privacidade diferencial foi de aproximadamente 0,987, com desvio padrão de 0,012, ao passo que os modelos com PD apresentaram média de 0,811 e desvio padrão de 0,139. Conforme mostrado na figura 7, *boxplots* comparando a acurácia obtida por cada arquitetura revelam que modelos treinados com PD atingem, em média, valores de acurácia inferiores.

Para investigar se essa diferença era estatisticamente significativa, foi realizado um teste de hipótese com base na acurácia dos modelos. A hipótese nula (H_0) considera que não há diferença entre as distribuições de acurácia dos modelos com e sem PD, enquanto a hipótese alternativa (H_1) propõe que os modelos treinados com PD apresentam acurácia estatisticamente inferior.

Devido à violação da normalidade e à diferença entre as variâncias dos grupos, optou-se pela utilização do teste não paramétrico de Wilcoxon, com hipótese alternativa unilateral.

O resultado do teste (W = 25, *p-valor* < $1{,}05 \times 10^{-12}$) rejeitou a hipótese nula com alto nível de significância, confirmando que a aplicação de privacidade diferencial reduz estatisticamente o desempenho dos modelos em termos de acurácia. Essa evidência reforça o compromisso entre privacidade e utilidade já teorizado na literatura, indicando que, embora a PD traga benefícios relevantes de proteção aos dados, ela também impõe limitações práticas ao desempenho dos modelos treinados.

Ainda assim, a Arquitetura B mostrou-se mais resiliente à perda de desempenho. Seu uso de técnicas de regularização como *group normalization* e *dropout* parece contribuir para maior estabilidade mesmo diante da adição de ruído nos gradientes. Esse comportamento está em linha com observações feitas por Lei Yu *et al.* [36] e NhatHai Phan *et al.* [26], que apontam que redes mais profundas e com mecanismos de regularização tendem a sofrer menos degradação sob PD.

5.5 Tempo de Treinamento

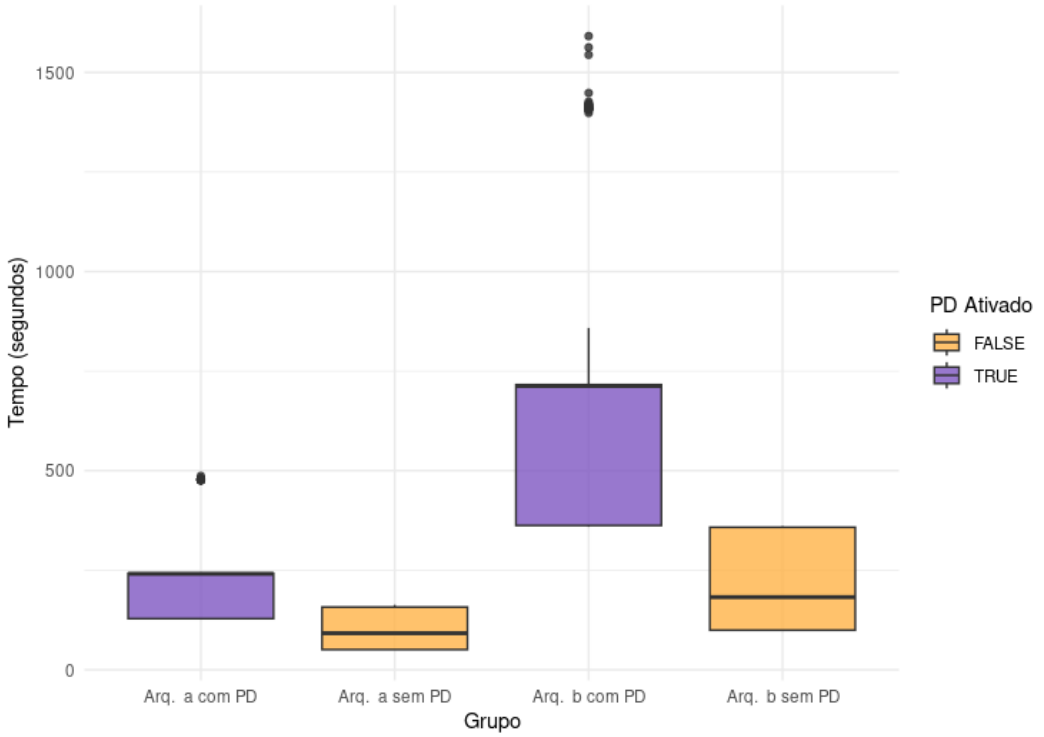

Figura 8: Gráfico do tempo de treinamento para as arquiteturas consideradas, com e sem PD.

Fonte: Elaboração própria.

A aplicação de PD também impacta diretamente o tempo de treinamento, conforme mostrado no gráfico de *boxplot* da Figura 8. Ao interpretar a figura, se torna evidente que os modelos com PD levam significativamente mais tempo para serem treinados. O custo computacional se eleva devido à necessidade de limitação dos gradientes individuais, aplicação de ruído gaussiano e ao controle do orçamento de privacidade ao longo das iterações. A aplicação de PD, sobretudo por meio da adição de ruído aos gradientes, parece aumentar o tempo de convergência da rede a um mínimo local da função de custo.

Esse aumento no tempo de execução pode inviabilizar a aplicação de PD em cenários em que a eficiência computacional seja crítica, especialmente quando o treinamento é realizado em dispositivos com poucos recursos.

5.6 Análise do Trade-off Privacidade vs Utilidade

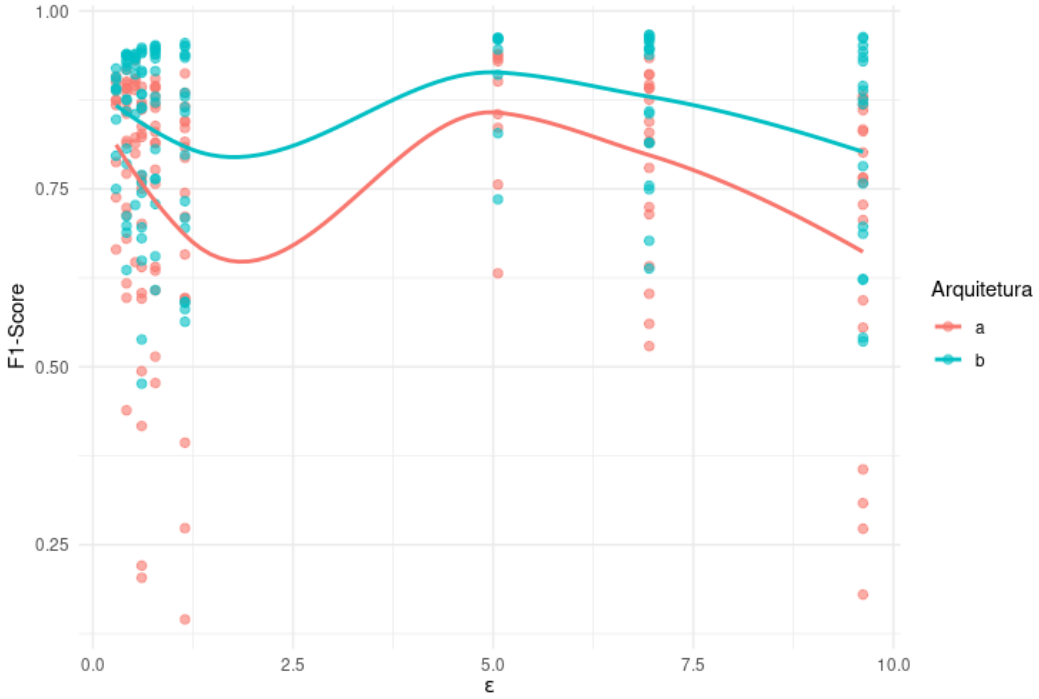

Figura 9: Gráfico de compromisso entre privacidade e utilidade.

Fonte: Elaboração própria.

A figura 9 apresenta uma análise do compromisso (*trade-off*) entre o nível de privacidade (medido por ϵ) e a utilidade do modelo (mensurada por *F1-score*). Como esperado, valores menores de ϵ — que indicam maior privacidade — tendem a resultar em menores *F1-scores*. Esse padrão confirma a relação inversamente proporcional entre privacidade e desempenho, já destacada em [4].

É possível observar, no entanto, que existe uma faixa intermediária de valores de ϵ (em torno de 3 a 6) em que o modelo ainda mantém níveis aceitáveis de desempenho, ao mesmo tempo em que oferece garantias razoáveis de privacidade. Essa faixa representa um ponto de equilíbrio interessante para aplicações práticas.

É importante destacar que o cálculo do parâmetro de privacidade ϵ, realizado automaticamente pela biblioteca Opacus, é derivado dos hiperparâmetros utilizados durante o treinamento, como o tamanho do batch, número de épocas, limitação de gradiente e multiplicador de ruído. Em função disso, os valores de ϵ não variam de forma contínua, mas assumem valores discretos dependentes dessas configurações. Essa característica reforça a necessidade de

planejamento cuidadoso dos hiperparâmetros, uma vez que escolhas distintas podem resultar em diferentes níveis de privacidade garantida.

6 Conclusão

Este capítulo apresenta as principais considerações finais decorrentes do desenvolvimento deste trabalho, bem como sugestões de direções futuras que podem orientar pesquisas subsequentes. Os tópicos abordam tanto as reflexões sobre os resultados obtidos quanto as limitações e possibilidades de aprimoramento das abordagens estudadas.

6.1 Considerações Finais

Este trabalho teve como objetivo investigar a aplicação da privacidade diferencial no contexto da AP, abordando tanto os fundamentos teóricos quanto a implementação prática e avaliação experimental de seus efeitos. Através do estudo do algoritmo DP-SGD, foi possível compreender como técnicas de limitação de gradiente e injeção de ruído podem ser integradas ao processo de treinamento de redes neurais para garantir proteção formal da privacidade dos dados.

Os experimentos realizados demonstraram de forma clara o compromisso entre privacidade e utilidade. À medida que aumentam as garantias de privacidade, há uma degradação perceptível no desempenho dos modelos, especialmente nas arquiteturas mais simples. Esse comportamento sugere que arquiteturas mais complexas e profundas, com camadas como *batch normalization* e *dropout*, podem contribuir para a robustez do modelo frente às perturbações introduzidas por mecanismos de privacidade.

Apesar de não existir um consenso de valores para os hiperparâmetros adicionados pelo uso de privacidade diferencial, houve uma faixa de valores de ϵ (entre 3 e 6) em que foi possível atingir uma bom equilíbrio entre privacidade e utilidade. Isso pode ser um bom indicativo para como encontrar um bom conjunto de valores para os hiperparâmetros adicionados pelo uso de PD.

Um grande ponto de destaque foi o aumento no tempo de treinamento ao se aplicar PD. O custo computacional adicional foi altíssimo, e isso afeta diretamente a viabilidade de aplicação dessas técnicas. Ainda assim, os resultados evidenciam que é possível alcançar um equilíbrio razoável entre proteção de dados e desempenho do modelo, desde que sejam escolhidas adequadamente as configurações de hiperparâmetros, como o multiplicador de ruído e o valor de imitação de gradiente.

Conclui-se que a privacidade diferencial tem o potencial de ser uma ferramenta valiosa para o desenvolvimento de sistemas de IA mais éticos, transparentes e alinhados às legislações de proteção de dados. No entanto, sua aplicação ainda impõe desafios técnicos importantes, sobretudo relacionados à calibragem dos parâmetros de privacidade e à manutenção da utilidade dos modelos.

6.2 Trabalhos Futuros

Técnicas adaptativas de ruído têm sido explorada por autores como Galen Andrew *et al.* [3] e Lei Yu *et al.* [36]. Eles propõem o ajuste dinâmico dos parâmetros de limitação de gradiente e da intensidade do ruído ao longo do treinamento, com o objetivo de minimizar perdas de desempenho. Essas abordagens ainda carecem de validação ampla na literatura.

A integração entre PD e FL, conforme sugerido por Brendan McMahan *et al.* [21] e Robin Geyer *et al.* [12], surge como uma estratégia promissora para ambientes descentralizados e sensíveis à privacidade, como aqueles que contêm dispositivos móveis e sistemas médicos, mas enfrenta desafios técnicos relevantes, como a coordenação eficiente e segura entre os dispositivos participantes.

A aplicação de PD em conjuntos de dados mais complexos, como imagens de alta resolução ou dados multimodais, permanece como uma necessidade prática não plenamente resolvida, dado que a maior parte dos estudos — incluindo este trabalho — utiliza bases simplificadas e bem estruturas em seus experimentos.

Afim de encontrar um bom ponto de equilíbrio (tal como o observado na Seção 5.6 para a obtenção de boas configurações de hiperparâmetros de futuros modelos, seria interessante buscar um ponto comum de equilíbrio entre privacidade e utilidade entre variados tipos de modelos e de dados. Isso pode constituir um grande desafio, porém seria de grande ajuda para a literatura da área de PD, podendo contribuir na ampliação de aplicação de PD em projetos e pesquisa globalmente.

Referências

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. *arXiv preprint arXiv:1607.00133*, 2016.

[2] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Springer, Cham, Switzerland, 1 edition, 2018.

[3] Galen Andrew and et al. Differentially private learning with adaptive clipping. *arXiv preprint arXiv:1905.03871*, 2019.

[4] Eugene Bagdasaryan and Vitaly Shmatikov. Differential privacy has disparate impact on model accuracy. *arXiv preprint arXiv:1905.12101*, 2019.

[5] Bruegel. The economic value of personal data for online platforms, firms and consumers. *Bruegel Blog*, 2020. Acesso em: 17 jul. 2025.

[6] Chris Deotte. How to choose cnn architecture mnist, 2020. Acessado em 17 de julho de 2025.

[7] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference*, pages 265–284. Springer, 2006.

[8] Cynthia Dwork and Aaron Roth. *The Algorithmic Foundations of Differential Privacy*, volume 9. Foundations and Trends in Theoretical Computer Science, 2014.

[9] Cynthia Dwork, Guy N. Rothblum, and Salil Vadhan. Boosting and differential privacy. In *Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, 2010.

[10] The Economist. The world's most valuable resource is no longer oil, but data, 2017. Acesso em: 17 jul. 2025.

[11] Úlfar Erlingnsson, Vasyl Pihur, and Aleksandra Korolova. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 21st ACM Con-*

ference on Computer and Communications Security (CCS), pages 1054–1067. ACM, 2014.

[12] Robin C. Geyer, Tassilo Klein, and Moin Nabi. Differentially private federated learning: A client level perspective. *arXiv preprint arXiv:1712.07557*, 2017.

[13] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 315–323, 2011.

[14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.

[15] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2672–2680, 2014.

[16] The Guardian. Revealed: 50 million facebook profiles harvested for cambridge analytica in major data breach, 2018. Accessed: 2025-05.

[17] IDC and Seagate. Data age 2025: The digitization of the world from edge to core. Technical report, Seagate Technology, 2018. White paper.

[18] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[19] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[20] Omnifunnel Marketing. From data to dollars: The monetization of personal information by meta and google, 2023. Acesso em: 17 jul. 2025.

[21] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.

[22] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.

[23] Arvind Narayanan and Vitaly Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, pages 111–125, 2008.

[24] Nicolas Papernot, Martin Abadi, Úlfar Erlingsson, Ian Goodfellow, and Kunal Talwar. Semi-supervised knowledge transfer for deep learning from private training data. In *International Conference on Learning Representations (ICLR)*, 2017.

[25] Nicolas Papernot, Shuang Song, Ilya Mironov, Aditi Raghunathan, Kunal Talwar, and Úlfar Erlingsson. Tempered sigmoid activations for differentially private deep learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[26] NhatHai Phan, Xintao Wu, Han Hu, and Dejing Dou. Adaptive laplace mechanism: Differential privacy preservation in deep learning. *IEEE BigData*, 2017.

[27] W. Nicholson Price II and I. Glenn Cohen. Privacy in the age of medical big data. *Nature Medicine*, 25(1):37–43, 2019. Acesso em: 17 jul. 2025.

[28] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.

[29] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2017.

[30] Congzheng Song, Thomas Ristenpart, and Vitaly Shmatikov. Machine learning models that remember too much. *arXiv preprint arXiv:1709.07886*, 2017. Mostram que modelos podem memorizar dados específicos sem comprometer precisão preditiva.

[31] Latanya Sweeney. Simple demographics often identify people uniquely. Technical report, Carnegie Mellon University, 2000.

[32] Reihaneh Torkzadehmahani, Peter Kairouz, Benedict Paten, and Ameet Talwalkar. Dp-cgan: Differentially private synthetic data and label generation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 0–0, 2019.

[33] Stacey Truex, Ling Liu, Jinwen Chow, Mehmet Emre Gursoy, and Lei Wei. A hybrid approach to privacy-preserving federated learning. In *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 1–11, 2019.

[34] European Union. Regulation (eu) 2016/679 – recital 26: Gdpr and identifiability, 2016. Accessed: 2025-05.

[35] Liyang Xie, Kai Lin, Shu Wang, Fei Wang, and Jun Zhou. Differentially private generative adversarial network. In *arXiv preprint arXiv:1802.06739*, 2018.

[36] Lei Yu, Liyang Liu, Yun Pu, and Xintao Wu. Differentially private model publishing for deep learning. *arXiv preprint arXiv:1904.02200*, 2019.

[37] Jun Zhang, Benjamin I.P. Rubinstein, and Chihao Wang. Posterior sampling for differentially private bayesian inference. *Journal of Machine Learning Research*, 18(29):1–43, 2018.